Reinforcement Learning-Based Routing for Industrial Vehicle

1. Introduction

This document specifies a simplified but realistic Reinforcement Learning (RL) formulation for optimizing the routing of an industrial vehicle (e.g., an AGV or similar machinery) inside a factory or warehouse. The goal is to define a project that is sufficiently complete for a Master-level subject while remaining feasible to implement and analyze within limited time.

The approach is based on tabular Q-learning, with a discrete state space and a small, well-structured action space. The main optimization objectives are:

- Minimize the **average time per job** (transport mission).
- Control and implicitly reduce the impact of **queue size** (system load) on performance.

2. Problem Formulation

2.1 Industrial Context

The system under study consists of a single industrial vehicle operating inside a plant. The plant is modeled as a set of zones or nodes (e.g., workstations, intersections, storage areas) connected by paths. The vehicle must execute transport jobs consisting of moving a load from a pickup location to a delivery location.

In this first version, each episode of the RL environment corresponds to a single job: starting from an initial position, the vehicle must navigate to the pickup node, automatically load, then navigate to the drop node, and complete the job. Constraints such as battery level and workload (queue level) are represented in a simplified, discrete way.

2.2 Objectives

The primary performance objectives are:

- **Average time per job**: reduce the number of steps (time units) required to complete each transport mission.
- **Queue impact**: incorporate the system load (queue level) into the reward to encourage policies that are more efficient when the system is busy.

Secondary aspects, such as detailed energy management and multi-job episodes, can be added in future extensions of the project.

3. State Space Design

The state is designed to be:

- **Compact**: few discrete components.
- **Informative**: captures the main elements influencing routing decisions.
- **Suitable for tabular Q-learning**: fully discrete and of moderate size.

3.1 State Variables

The state at time *t* is defined as:

```
s_t = (location_id, battery_level, queue_level,
                    load_state)
```

with the following components:

- **location_id**: integer index of the current zone or node in the plant graph.
  Domain: `location_id ∈ {0, 1, ..., N−1}`, where N is the number of nodes.
- **battery_level**: discretized battery/fuel level of the vehicle.
  Domain: `battery_level ∈ {0, 1, 2}`, where 0 = low, 1 = medium, 2 = high.
- **queue_level**: discretized measure of system load or task queue.
  Domain: `queue_level ∈ {0, 1, 2}`, where 0 = empty/low load, 1 = moderate load, 2 = high load.
- **load_state**: binary indicator of whether the vehicle is currently carrying a load.
  Domain: `load_state ∈ {0, 1}`, where 0 = empty, 1 = loaded.

3.2 Treatment of Obstacles and Congestion

To keep the state space small, obstacles and local congestion are **not** explicitly encoded as state variables in this first version. Instead, they are represented implicitly in the environment dynamics and reward function. For example, attempting to traverse a blocked edge can result in no movement and a penalty in the reward, or a higher step cost.

3.3 State Space Size

Assuming a modest plant map with:

- N = 20 locations (nodes)
- 3 battery levels
- 3 queue levels
- 2 load states

The total number of possible states is:

$$|S| = 20 \times 3 \times 3 \times 2 = 360$$

This cardinality is very manageable for a tabular Q-learning implementation in a Master-level project.

4. Action Space Design

4.1 Movement Actions

The plant layout is represented as a graph whose nodes are `location_id` values. Each node has a set of neighboring nodes reachable via direct paths.

The **action** at each time step consists of choosing one of the neighboring nodes to move to:

- **Action**: "move to neighbor node *j*".

Formally, for a node `i` with neighbors `Neigh(i)`, the action set available in a state with `location_id = i` is:

$$A(i) = \{ \text{move\_to\_}j \mid j \in \text{Neigh}(i) \}$$

There is no explicit "wait" or "stay" action in this first version, to keep the action space minimal and the behavior focused on navigation.

4.2 Task and Charging Logic

To avoid an explosion in the number of actions, task-related decisions (pick up, drop, charge) are handled **automatically** by the environment:

- If the vehicle arrives at the **pickup node** while empty (`load_state = 0`), the environment automatically sets `load_state = 1`.
- If the vehicle arrives at the **drop node** while loaded (`load_state = 1`), the environment completes the job, gives a positive reward, and resets `load_state = 0` and ends the episode.
- If a charging or maintenance node is used in later extensions, its behavior can also be automatically triggered when the vehicle visits that node.

This design keeps the focus on routing and simplifies the implementation.

5. Episode Definition and Environment Dynamics

5.1 Episode Definition

An **episode** represents the execution of a single transport job. Each episode includes:

- A **pickup node** `pickup_node`.
- A **drop node** `drop_node`.
- An initial state `s_0` for the vehicle.

The episode starts when the job is generated and the vehicle is placed in its initial state. It ends when the job is successfully completed or a failure condition occurs.

5.2 Episode Initialization

At the start of each episode, the environment performs:

- Selects a `start_node` (e.g., fixed or randomly chosen).
- Selects a `pickup_node` and `drop_node` (scenarios can be predefined or sampled).
- Sets `load_state = 0` (vehicle initially empty).
- Sets `battery_level`, typically to a high level (e.g., 2).
- Sets `queue_level`, either fixed for the episode or sampled from {0, 1, 2}.

The initial state is then:

```
s_0 = (start_node, battery_level_initial,
       queue_level_initial, 0)
```

5.3 Step Dynamics

At each time step *t* within an episode:

1. The agent observes state `s_t = (location_id, battery_level, queue_level, load_state)`.
2. The agent chooses an action `a_t`: move to one of the neighbor nodes of `location_id`.
3. The environment executes the following logic:
    - **Movement**:
      If the chosen action corresponds to a valid neighbor, the vehicle moves to that neighbor: `location_id' = neighbor_node`. If the action is invalid (ideally filtered out), the environment can ignore it, keep the location unchanged, and apply a penalty.
    - **Time progression**:
      Each step corresponds to one time unit. A fixed step cost is applied to the reward.

- **Queue dynamics**:
  In the simplest version, `queue_level` is kept constant during the episode. More advanced versions can update it as jobs are completed or as time passes.
- **Battery dynamics**:
  Battery can be updated with a simple rule (e.g., decrement after a certain number of steps). In the initial project version, battery dynamics can be simplified or even kept constant, as energy is not the primary optimization target yet.
- **Load and job completion**:
  If `location_id' == pickup_node` and `load_state == 0`, then set `load_state' = 1` (automatic loading).
  If `location_id' == drop_node` and `load_state == 1`, then the job is completed: apply a positive reward and mark the episode as done.

4. The environment computes the total reward `r_t` for this transition and returns the next state `s_{t+1}`, the reward `r_t`, and a flag `done` indicating whether the episode has ended.

6. Reward Function Design

6.1 Objectives Reflected in the Reward

The reward is designed to reflect mainly two objectives:

- Minimization of **time per job** (number of steps).
- Incorporation of **queue level** into the penalty, to prefer efficient operation when the system is busy.

Battery constraints and energy consumption are modeled at a basic level and can be enhanced later.

6.2 Reward Components

At each time step, the reward `r_t` is composed of:

```
r_t = r_step + r_queue + r_task_completion + r_failure
```

with the following components (initial choice of parameters):

- **Step penalty**:
  `r_step = −1`
  A fixed negative reward per step encourages shorter paths and faster job completion.
- **Queue penalty**:
  `r_queue = −0.3 × queue_level`
  A small penalty proportional to the queue level encourages the agent to be especially efficient when the system is more loaded.
- **Task completion reward**:
  `r_task_completion = +20` (applied once when the job is successfully completed).

This large positive reward compensates for the per-step penalties and defines the main goal of the episode.

- **Failure penalty**:
  `r_failure = −30` (applied when a critical failure occurs, for instance a battery-related failure or an illegal/blocked action if such events are modeled).
  This discourages behaviors that lead to dead ends or system downtime.

The numeric values are initial design choices; they can be tuned empirically by observing training curves and qualitative behavior of the agent.

7. Q-Learning Formulation

7.1 Q-Table Structure

The RL algorithm used is **tabular Q-learning**. The Q-table stores an estimate of the value `Q(s, a)` for every state-action pair. With the chosen state representation, the table can be indexed as:

```
Q[location_id][battery_level][queue_level][load_state]
                        [action]
```

where `action` refers to one of the valid neighbors of the current node (or, in implementation, to an index in a finite action set filtered by neighborhood).

7.2 Q-Learning Update Rule

After observing a transition `(s_t, a_t, r_t, s_{t+1})`, the Q-value is updated as:

```
Q(s_t, a_t) ← Q(s_t, a_t) + α × ( r_t + γ × max_{a'}
            Q(s_{t+1}, a') − Q(s_t, a_t) )
```

where:

- `α` is the learning rate (e.g., 0.1).
- `γ` is the discount factor for future rewards (e.g., 0.95).
- `max_{a'} Q(s_{t+1}, a')` is the estimated value of the best action in the next state.

7.3 Exploration Policy (ε-Greedy)

During training, an **ε-greedy** policy is used:

- With probability `ε`, choose a random valid action (exploration).
- With probability `1 − ε`, choose the action that maximizes the current Q-value (exploitation).

The parameter ε can be decayed over time, for example starting from 1.0 and decreasing gradually to a smaller value such as 0.1 or 0.05, to favor exploration at the beginning and exploitation later in training.

8. Training Loop and Environment Interface

8.1 Environment Interface

The environment is conceptually defined by two methods:

- **`reset()`**: initializes a new episode (job) and returns the initial state `s_0`.
- **`step(action)`**: applies the given action, updates the environment and agent state, and returns:
  - next state `s_{t+1}`
  - reward `r_t`
  - boolean `done` indicating if the episode has terminated
  - optional `info` field for diagnostics

8.2 High-Level Training Loop

A high-level training loop for the Q-learning agent is:

```
initialize Q(s, a) arbitrarily (e.g., zeros)

for episode in 1..num_episodes:

    s = env.reset()         # initial state
    done = False

    while not done:

        # ε-greedy action selection
        if random_uniform(0,1) < epsilon:
            a = random_valid_action(s)
        else:
            a = argmax_a Q[s, a]

        # environment transition
        s_next, r, done, info = env.step(a)

        # Q-learning update
        best_next_Q = max_{a'} Q[s_next, a']
        Q[s, a] = Q[s, a] + alpha * (r + gamma * best_next_Q - Q[s, a])

        # move to next state
        s = s_next

    # optional: decay epsilon
    epsilon = max(epsilon_min, epsilon * epsilon_decay)
```

This loop is language-agnostic and can be implemented in Python or any other language suitable for the project.

9. Evaluation Metrics and Experimental Plan

9.1 Metrics

After training, the learned policy should be evaluated with a **greedy** strategy ($\varepsilon = 0$). Typical metrics include:

- **Average job duration**: average number of steps required to complete a job.
- **Average total reward per episode**: aggregate of rewards over an episode.
- **Failure rate**: proportion of episodes ending with a failure (if failure modes are implemented).
- **Queue-level sensitivity**: comparison of performance across different initial or fixed `queue_level` values.

9.2 Experimental Procedure

A typical experimental plan may include:

- Train the agent for a given number of episodes (e.g., several thousands) on a fixed plant layout and job distribution.
- Periodically evaluate the policy (without exploration) and track the metrics listed above.
- Visualize learning curves, such as:
    - Average episode reward vs. training episodes.
    - Average job duration vs. training episodes.
- Analyze qualitatively the learned routing behavior: for example, shortest paths, behavior under high queue levels, and handling of battery constraints if modeled.

10. Possible Extensions

Once the base project is implemented and analyzed, several extensions are possible:

- **Energy-aware routing**: incorporate energy consumption into the reward, model charging stations explicitly, and add incentives or penalties related to battery level.
- **Multiple jobs per episode**: allow the agent to execute a sequence of transport jobs and maintain a non-trivial queue dynamics across time.
- **More detailed plant layout**: increase N (number of nodes) and refine the topology to better represent a real factory or warehouse.
- **Stochastic disturbances**: model obstacles, congestion, or travel times as stochastic variables.
- **Function approximation**: for larger state spaces, replace tabular Q-learning with Deep Q-Networks (DQN) or other function approximators.

11. Summary of Design Decisions from the Chat

The design summarized in this document is the result of an iterative step-by-step discussion and refinement process. The key decisions made are:

- **Use discrete zones** (nodes) for locations instead of continuous coordinates, for ease of implementation and clear interpretation.
- **Discretize battery and queue levels** into three levels each, and model load as a binary variable, yielding a compact state space of manageable size (around 360 states for 20 nodes).
- **Exclude obstacles from the explicit state** in the first version, handling them implicitly in the environment transitions and reward if needed.
- **Define actions as movements to neighbor nodes only**, avoiding additional task-related actions and making the environment responsible for automatic loading/unloading at the corresponding nodes.
- **One job per episode** as a clear and simple episodic structure, ideal for didactic purposes.
- **Reward focused on time per job and queue influence**, with:
    - $r_{step} = -1$
    - $r_{queue} = -0.3 \times queue\_level$
    - $r_{task\_completion} = +20$
    - $r_{failure} = -30$
- **Use of standard tabular Q-learning** with an ε-greedy exploration strategy, learning rate $\alpha \sim 0.1$, and discount factor $\gamma \sim 0.95$.

This specification provides a solid foundation for implementing the RL-based routing project, analyzing its behavior, and extending it in future work if additional complexity is desired.